# Lecture 05.13: Types of Inheritance & C3 Linearization (MRO)

## 📌 Executive Summary

Single, multiple, multilevel, hierarchical, and hybrid inheritance; the Diamond Problem and C3 Linearization MRO algorithm.

---

## 🔬 Theoretical Foundations & Principles

### 1. Taxonomy of Inheritance Structures

- **Single:** Child derives from one Parent.
- **Multilevel:** A $\rightarrow$ B $\rightarrow$ C inheritance chain.
- **Hierarchical:** Multiple Children derive from one Parent.
- **Multiple:** One Child derives from multiple Parents.
- **Hybrid:** Combination forming diamond graphs.

### 2. The Diamond Problem & C3 Linearization

When multiple inheritance paths converge on a common ancestor, Python resolves method ordering using the **C3 Linearization** algorithm, guaranteeing:

1. Local precedence order (parents listed left-to-right).
2. Monotonicity (subclass precedes superclass).

$$L(C) = C + \text{merge}(L(B_1), L(B_2), \ldots, L(B_k), B_1 B_2 \ldots B_k)$$

---

## 💻 Production Implementation & Code

```
class Herbivore:
    def feed(self):
        return "Consumes plants & berries"

class Carnivore:
    def hunt(self):
```

```
        return "Hunts fish & small mammals"

class Bear(Herbivore, Carnivore):
    """Multiple inheritance: Omnivore entity."""
    pass

b = Bear()
print("Bear Diet:", b.feed(), "|", b.hunt())
print("Bear MRO:", [cls.__name__ for cls in Bear.mro()])
```

---

## ⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.